Assignment

Bachelor of Computer Applications

Semester III

Database Management System

Topic: IMPLEMENTATION OF DATA INTEGRITY CONSTRAINTS

BY

Name: M Blessy

Registration Number: 2511021240067

DEPARTMENT OF COMPUTER APPLICATIONS

Alliance university

Chikkahagade cross, Anekal, Bangaluru 562106

IMPLEMENTATION OF DATA INTEGRITY CONSTRAINTS

1. Objective

To implement and verify data integrity constraints in the Hospital Management System database using SQL. The constraints include Primary Key, Foreign Key, UNIQUE, CHECK, and DEFAULT constraints. They help maintain accurate, valid, and consistent data and establish relationships between tables.

2. Introduction

Data integrity means maintaining the accuracy, consistency, and reliability of data stored in a database. In a Hospital Management System, patient details, doctor information, appointments, admissions, prescriptions, medicines, laboratory tests, and bills must be stored correctly.

For example, an appointment should not refer to a patient who does not exist in the Patient table. Similarly, medicine prices should not be negative, and two departments should not have the same department name.

SQL integrity constraints prevent invalid data from being inserted or updated and help maintain proper relationships between tables.

3. Primary Key Constraints

A **Primary Key** uniquely identifies every record in a table. It does not allow duplicate or NULL values.

In the Hospital Management System, each of the 11 tables has its own primary key.

Table	Primary key
Department	department_id
Doctor	doctor_id
Patient	patient_id
Appointment	appointment_id
Room	room_id
Admission	admission_id
Prescription	prescription_id
Medicine	medicine_id

Prescription_Detail	prescription_detail_id
Lab_Test	test_id
Bill	bill_id

Example SQL:

```
CREATE TABLE Patient (
   patient_id INT PRIMARY KEY,
   name VARCHAR(60) NOT NULL,
   dob DATE,
   gender VARCHAR(10),
   phone_no VARCHAR(15),
   address VARCHAR(150),
   blood_group VARCHAR(5),
   emergency_contact VARCHAR(15)
);
```

Purpose: The patient_id uniquely identifies each patient and prevents two patients from having the same ID.

Note: This is an example of the existing table definition. The table has already been created in the database.

4. Foreign Key Constraints

A **Foreign Key** connects one table to another by referring to a primary key or another eligible unique key. It prevents a record from referring to a non-existent record in the referenced table.

The Hospital Management System uses foreign keys to maintain relationships between departments, doctors, patients, appointments, admissions, prescriptions, medicines, laboratory tests, and bills.

Table	Foreign key	References
Doctor	department_id	Department(department_id)
Department	head_doctor_id	Doctor(doctor_id)
Appointment	patient_id	Patient(patient_id)
Appointment	doctor_id	Doctor(doctor_id)
Admission	patient_id	Patient(patient_id)
Admission	room_id	Room(room_id)
Admission	doctor_id	Doctor(doctor_id)
Prescription	appointment_id	Appointment(appointment_id)
Prescription	patient_id	Patient(patient_id)

Prescription	doctor_id	Doctor(doctor_id)
Prescription_Detail	prescription_id	Prescription(prescription_id)
Prescription_Detail	medicine_id	Medicine(medicine_id)
Lab_Test	patient_id	Patient(patient_id)
Lab_Test	doctor_id	Doctor(doctor_id)
Bill	patient_id	Patient(patient_id)
Bill	appointment_id	Appointment(appointment_id)
Bill	admission_id	Admission(admission_id)

Example SQL:

```
CONSTRAINT fk_appointment_patient
   FOREIGN KEY (patient_id)
   REFERENCES Patient(patient_id)
```

Purpose: This constraint ensures that an appointment refers to an existing patient. A foreign key can be NULL when the column permits NULL and no value is supplied.

5. UNIQUE Constraints

A **UNIQUE constraint** prevents duplicate values in a column or combination of columns.

The following UNIQUE constraints are implemented in the database:

Table	Column	Purpose
Room	room_number	Prevents the same room number from being assigned to multiple room records.
Department	department_name	Prevents duplicate department names.
Doctor	email	Prevents duplicate doctor email addresses.

SQL statements used:

```
ALTER TABLE Department
ADD CONSTRAINT uq_department_name
UNIQUE (department_name);

ALTER TABLE Doctor
ADD CONSTRAINT uq_doctor_email
UNIQUE (email);
```

The Room table already contains the following constraint in its table definition:

CONSTRAINT uq_room_number UNIQUE (room_number)

Purpose: These constraints help avoid duplicate department names, doctor email addresses, and room numbers. In MySQL, a UNIQUE constraint can permit multiple NULL values, depending on the column and database rules.

6. CHECK Constraints

A **CHECK constraint** ensures that values entered into a column satisfy a specified condition.

The following CHECK constraints are implemented in the Hospital Management System:

Constraint	Condition	Purpose
chk_doctor_fee	consultation_fee >= 0	Prevents negative consultation fees.
chk_doctor_experience	experience_years >= 0	Prevents negative years of experience.
chk_room_charge	daily_charge >= 0	Prevents negative room charges.
chk_room_status	status IN ('Available', 'Occupied', 'Maintenance')	Restricts room status to valid values.
chk_appointment_status	status IN ('Scheduled', 'Completed', 'Cancelled')	Restricts appointment status to valid values.
chk_admission_status	status IN ('Admitted', 'Discharged')	Restricts admission status to valid values.
chk_bill_amount	total_amount >= 0	Prevents negative bill amounts.
chk_payment_status	payment_status IN ('Pending', 'Paid', 'Failed')	Restricts payment status to valid values.
chk_lab_status	status IN ('Pending', 'Completed', 'Cancelled')	Restricts laboratory test status to valid values.
chk_medicine_price	unit_price >= 0	Prevents negative medicine prices.
chk_medicine_stock	stock_quantity >= 0	Prevents negative medicine stock quantities.
chk_prescription_duration	duration_days > 0	Ensures prescription duration is positive.

chk_prescription_quantity	quantity > 0	Ensures prescription quantity is positive.

Example SQL statements:

```
ALTER TABLE Doctor
ADD CONSTRAINT chk_doctor_fee
CHECK (consultation_fee >= 0);

ALTER TABLE Medicine
ADD CONSTRAINT chk_medicine_stock
CHECK (stock_quantity >= 0);

ALTER TABLE Prescription_Detail
ADD CONSTRAINT chk_prescription_duration
CHECK (duration_days > 0);
```

Purpose: CHECK constraints prevent invalid values, such as negative medicine stock, negative consultation fees, or zero-day prescription durations.

7. DEFAULT Constraints

A **DEFAULT constraint** automatically supplies a predefined value when an INSERT statement does not provide a value for that column.

The following DEFAULT constraints are implemented in the database:

Table	Column	Default value	Purpose
Appointment	status	'Scheduled'	Sets a new appointment's initial status to Scheduled when omitted.
Room	status	'Available'	Sets a room's initial status to Available when omitted.
Admission	status	'Admitted'	Sets a new admission's initial status to Admitted when omitted.
Bill	payment_status	'Pending'	Sets a new bill's initial payment status to Pending when omitted.

Example SQL:

```
status VARCHAR(15) DEFAULT 'Scheduled'
```

Purpose: DEFAULT constraints reduce the need to enter common initial values manually and help maintain consistent records.

8. Verification of Data Integrity Constraints

The following SQL statements can be executed in MySQL Workbench to verify the constraints and relationships in the existing database.

8.1 Verify all tables and constraints

```
USE Hospital_Management_System;

SELECT
   TABLE_NAME,
   CONSTRAINT_NAME,
   CONSTRAINT_TYPE
FROM information_schema.TABLE_CONSTRAINTS
WHERE CONSTRAINT_SCHEMA = 'Hospital_Management_System'
ORDER BY TABLE_NAME, CONSTRAINT_TYPE;
```

8.2 Verify foreign key relationships

```
SELECT
   TABLE_NAME,
   COLUMN_NAME,
   CONSTRAINT_NAME,
   REFERENCED_TABLE_NAME,
   REFERENCED_COLUMN_NAME
FROM information_schema.KEY_COLUMN_USAGE
WHERE TABLE_SCHEMA = 'Hospital_Management_System'
  AND REFERENCED_TABLE_NAME IS NOT NULL
ORDER BY TABLE_NAME, COLUMN_NAME;
```

8.3 Verify UNIQUE and CHECK constraints

```
SELECT
   TABLE_NAME,
   CONSTRAINT_NAME,
   CONSTRAINT_TYPE
FROM information_schema.TABLE_CONSTRAINTS
WHERE CONSTRAINT_SCHEMA = 'Hospital_Management_System'
  AND CONSTRAINT_TYPE IN ('UNIQUE', 'CHECK')
ORDER BY TABLE_NAME, CONSTRAINT_TYPE;
```

TABLE_NAME	CONSTRAINT_NAME	CONSTRAINT_TYPE
admission	chk_admission_status	CHECK
admission	fk_admission_doctor	FOREIGN KEY
admission	fk_admission_patient	FOREIGN KEY
admission	fk_admission_room	FOREIGN KEY
admission	PRIMARY	PRIMARY KEY
appointment	chk_appointment_status	CHECK
appointment	fk_appointment_doctor	FOREIGN KEY
appointment	fk_appointment_patient	FOREIGN KEY
appointment	PRIMARY	PRIMARY KEY
bill	chk_bill_amount	CHECK
bill	chk_payment_status	CHECK
bill	fk_bill_admission	FOREIGN KEY
bill	fk_bill_appointment	FOREIGN KEY
bill	fk_bill_patient	FOREIGN KEY
bill	PRIMARY	PRIMARY KEY
department	fk_department_head_d...	FOREIGN KEY
department	PRIMARY	PRIMARY KEY
department	uq_department_name	UNIQUE
doctor	chk_doctor_fee	CHECK
doctor	chk_doctor_experience	CHECK
doctor	fk_doctor_department	FOREIGN KEY
doctor	PRIMARY	PRIMARY KEY
doctor	uq_doctor_email	UNIQUE
lab_test	chk_lab_status	CHECK
lab_test	fk_lab_doctor	FOREIGN KEY
lab_test	fk_lab_patient	FOREIGN KEY
lab_test	PRIMARY	PRIMARY KEY
medicine	chk_medicine_price	CHECK
medicine	chk_medicine_stock	CHECK
medicine	PRIMARY	PRIMARY KEY
patient	PRIMARY	PRIMARY KEY
prescription	fk_prescription_appoint...	FOREIGN KEY
prescription	fk_prescription_doctor	FOREIGN KEY
prescription	fk_prescription_patient	FOREIGN KEY
prescription	PRIMARY	PRIMARY KEY
prescription...	chk_prescription_duration	CHECK
prescription...	chk_prescription_quantity	CHECK
prescription...	fk_detail_medicine	FOREIGN KEY
prescription...	fk_detail_prescription	FOREIGN KEY
prescription...	PRIMARY	PRIMARY KEY
room	chk_room_charge	CHECK
room	chk_room_status	CHECK
room	PRIMARY	PRIMARY KEY
room	uq_room_number	UNIQUE

8.4 Verify DEFAULT values

```
SELECT
   TABLE_NAME,
   COLUMN_NAME,
   COLUMN_DEFAULT
FROM information_schema.COLUMNS
WHERE TABLE_SCHEMA = 'Hospital_Management_System'
```

```
  AND COLUMN_DEFAULT IS NOT NULL
ORDER BY TABLE_NAME, COLUMN_NAME;
```

TABLE_NAME	COLUMN_NAME	COLUMN_DEFAULT
admission	status	Admitted
appointment	status	Scheduled
bill	payment_status	Pending
lab_test	status	Pending
room	status	Available

8.5 Verify relationships using JOIN

The following query checks whether appointment records are linked to existing patient and doctor records.

```
SELECT
   a.appointment_id,
   p.name AS patient_name,
   d.name AS doctor_name,
   a.appointment_date,
   a.status
FROM Appointment a
JOIN Patient p
   ON a.patient_id = p.patient_id
JOIN Doctor d
   ON a.doctor_id = d.doctor_id;
```

appointment_id	patient_name	doctor_name	appointment_date	status
9002	Test Patient	Dr. Arun	2026-10-25	Scheduled
7001	Aarav Mehta	Priya Sharma	2026-10-26	Scheduled
7004	Meera Joseph	Priya Sharma	2026-10-28	Scheduled
7002	Diya Nair	Rahul Kumar	2026-10-26	Scheduled
7003	Kiran Kumar	Ananya Rao	2026-10-27	Completed

This query demonstrates the relationships between the Appointment, Patient, and Doctor tables.

9. Expected Integrity Behaviour

The following examples explain how the constraints protect the database. These are examples of expected behaviour; they should not be executed against the existing data unless testing is specifically required.

Test case	Expected result
Insert a duplicate primary key	The database rejects the duplicate key.
Insert an appointment referring to a non-existent patient	The foreign key rejects the record.
Insert a duplicate department name	The UNIQUE constraint rejects the duplicate.

Insert a negative consultation fee	The CHECK constraint rejects the invalid value.
Insert a negative medicine stock quantity	The CHECK constraint rejects the invalid value.
Insert an appointment without specifying its status	The DEFAULT value is 'Scheduled', provided the column is omitted from the INSERT statement.
Insert a bill without specifying payment status	The DEFAULT value is 'Pending', provided the column is omitted from the INSERT statement.

10. Conclusion

The Hospital Management System uses Primary Key, Foreign Key, UNIQUE, CHECK, and DEFAULT constraints to maintain data integrity. Primary keys uniquely identify records, foreign keys establish relationships between tables, UNIQUE constraints prevent duplicate values, CHECK constraints restrict invalid data, and DEFAULT constraints provide predefined values.

The verification queries help confirm that the constraints are present and that related records can be retrieved through joins. These measures improve the accuracy, consistency, and reliability of the hospital database.